

DOCUMENTAÇÃO DE PROJETO

Amazon EC2

Provisionamento de Infraestrutura via Console e AWS CLI — Host Bastion e Servidor Web

Parâmetro	Detalhe
Serviço principal	Amazon EC2
Métodos utilizados	Console de Gerenciamento da AWS + AWS CLI
Ambiente	VPC dedicada com sub-redes pública e privada
Duração aproximada	45 minutos

1. Problema / Contexto

Ambientes de nuvem exigem dois perfis distintos de provisionamento: a agilidade do console gráfico para decisões pontuais e a repetibilidade da linha de comando para ambientes produtivos e pipelines automatizados. Sem dominar ambas as abordagens, qualquer time de infraestrutura fica exposto — ou à lentidão de processos manuais não escaláveis, ou à caixa-preta de ferramentas que ninguém sabe operar quando o pipeline falha.

Além disso, um padrão recorrente em ambientes sem maturidade de segurança é a configuração de credenciais estáticas diretamente nas instâncias para que elas possam chamar APIs da AWS — prática que cria superfícies de ataque desnecessárias e dificulta a rotação de acessos. Era necessário demonstrar, na prática, uma alternativa robusta a esse antipadrão.

2. Objetivo

Meu objetivo técnico central foi dominar os dois métodos de provisionamento de instâncias EC2 e entender, de forma prática, quando cada abordagem é a escolha correta. De forma mais específica, busquei:

- Provisionar um host bastion pelo Console de Gerenciamento da AWS, exercitando cada decisão de configuração de forma consciente.
- Utilizar esse bastion como ponto de execução de AWS CLI para provisionar uma segunda instância — o servidor web — de forma totalmente automatizada por script.
- Configurar a instância com um perfil IAM, eliminando por completo a necessidade de credenciais estáticas na máquina.
- Executar e validar um fluxo de troubleshooting real em uma instância com falhas propositalis, exercitando diagnóstico sistemático.

3. Solução

A solução foi estruturada em duas camadas: a criação manual e consciente do host bastion pelo Console, e a automação completa do servidor web via CLI a partir desse bastion. Essa arquitetura reproduz, em escala reduzida, o padrão Jump Server amplamente utilizado em ambientes corporativos para isolar o acesso administrativo da infraestrutura de produção.

3.1 Host Bastion — Console de Gerenciamento

Criei a primeira instância pelo Console da AWS, configurando cada parâmetro manualmente. A decisão mais relevante foi associar o perfil IAM Bastion-Role à instância em vez de configurar credenciais via `aws configure`. Isso garante que a instância receba permissões temporárias automaticamente via metadados, sem nenhuma chave estática gravada em disco.

Configurações definidas para o host bastion:

Parâmetro	Valor
AMI	Amazon Linux 2 (HVM) — Quick Start
Tipo de instância	t3.micro
Sub-rede	Sub-rede pública da Lab VPC
IP público automático	Habilitado
Grupo de segurança	Bastion security group (SSH habilitado)
Perfil IAM	Bastion-Role
Par de chaves	Não necessário (acesso via EC2 Instance Connect)

3.2 Acesso ao Bastion — EC2 Instance Connect

O acesso ao host bastion foi feito pelo EC2 Instance Connect, que abre um terminal diretamente no navegador sem necessidade de cliente SSH externo ou arquivo de chave privada. O serviço injeta uma chave SSH temporária na instância via API da AWS — essa chave expira em 60 segundos, tornando a janela de exposição mínima. O acesso é controlado integralmente por políticas IAM e registrado no CloudTrail.

3.3 Servidor Web — Provisionamento por AWS CLI

Com o terminal do bastion ativo, executei uma sequência de comandos para coletar dinamicamente todos os parâmetros necessários e lançar o servidor web sem nenhum valor hardcoded no script. Essa abordagem torna o processo portátil entre contas e regiões.

Etapa 1 — Determinar região e buscar o ID da AMI mais recente via Parameter Store:

```
# Detectar zona de disponibilidade e exportar a região automaticamente
AZ=`curl -s http://169.254.169.254/latest/meta-data/placement/availability-
```

```
zone`
export AWS_DEFAULT_REGION=${AZ::-1}

# Buscar o ID da AMI mais recente do Amazon Linux 2 via Parameter Store
AMI=$(aws ssm get-parameters \
  --names /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2 \
  --query 'Parameters[0].[Value]' --output text)
```

Etapa 2 — Recuperar dinamicamente o ID da sub-rede e do grupo de segurança:

```
# Sub-rede pública identificada pela tag Name
SUBNET=$(aws ec2 describe-subnets \
  --filters 'Name=tag:Name,Values=Public Subnet' \
  --query Subnets[].SubnetId --output text)

# Grupo de segurança identificado pelo nome
SG=$(aws ec2 describe-security-groups \
  --filters Name=group-name,Values=WebSecurityGroup \
  --query SecurityGroups[].GroupId --output text)
```

Etapa 3 — Lançar a instância com User Data para instalação automática do Apache:

```
INSTANCE=$( \
  aws ec2 run-instances \
  --image-id $AMI \
  --subnet-id $SUBNET \
  --security-group-ids $SG \
  --user-data file:///home/ec2-user/UserData.txt \
  --instance-type t3.micro \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Web Server}]' \
  --query 'Instances[*].InstanceId' \
  --output text \
)

# Monitorar o status até a instância estar em execução
aws ec2 describe-instances --instance-ids $INSTANCE \
  --query 'Reservations[].Instances[].State.Name' --output text

# Obter o DNS público para validação no navegador
aws ec2 describe-instances --instance-ids $INSTANCE \
  --query 'Reservations[].Instances[].PublicDnsName' --output text
```

Decisão arquitetural: Recuperar AMI ID, Subnet ID e Security Group ID dinamicamente via CLI em vez de hardcodá-los garante que o script funcione em qualquer conta ou região sem modificação. É o mesmo princípio que separa um script de laboratório de um script de produção.

4. Ferramentas

Amazon EC2	AWS CLI	EC2 Instance Connect
AWS IAM (Instance Profile)	AWS Systems Manager (Parameter Store)	Linux / Bash
Apache HTTP Server	User Data (bootstrapping)	Amazon CloudTrail

5. Resultado

A arquitetura foi entregue e validada com sucesso. O host bastion foi provisionado pelo Console e se tornou o ponto de execução da automação via CLI. O servidor web foi criado integralmente por linha de comando, sem nenhuma interação manual com o Console após o bastion estar ativo.

Ao acessar o DNS público da instância no navegador, a aplicação web foi exibida corretamente — confirmando que o script de User Data executou com sucesso na inicialização, instalando o Apache e implantando a aplicação sem intervenção manual.

Adicionalmente, os dois desafios de troubleshooting foram concluídos com sucesso, restaurando o funcionamento completo de uma instância propositalmente configurada com falhas — exercitando diagnóstico sistemático em condições próximas de um incidente real.

6. Aprendizados

Esta foi a seção de maior densidade técnica do projeto. Os aprendizados mais relevantes vieram dos momentos em que algo não funcionou como esperado — e não das etapas que executaram sem atrito.

6.1 Troubleshooting: porta fechada no grupo de segurança

A instância Misconfigured Web Server falhava silenciosamente ao tentar conexão via EC2 Instance Connect. O primeiro impulso seria reiniciar a instância ou revisar o estado do serviço — mas o diagnóstico correto exigiu subir uma camada antes: verificar se o tráfego sequer chegava à instância.

Sintoma	Causa raiz e correção
EC2 Instance Connect recusava a conexão sem mensagem de erro clara.	O grupo de segurança não possuía regra de entrada para a porta 22 (SSH). A adição da regra restaurou o acesso imediatamente.

Aprendizado: em qualquer falha de conectividade na AWS, o diagnóstico começa pelo grupo de segurança e pelas ACLs de rede — antes de inspecionar qualquer serviço dentro da instância. Essa ordem evita horas de depuração no lugar errado.

6.2 Troubleshooting: serviço não iniciado na inicialização

Com o acesso restabelecido, o servidor web ainda não respondia no navegador. O grupo de segurança desta vez permitia o tráfego HTTP. O problema estava dentro da instância.

Sintoma DNS público acessível, mas navegador retornava timeout sem carregar a página.	Causa raiz e correção O serviço Apache (httpd) não estava em execução. Iniciado manualmente com <code>sudo systemctl start httpd</code> , a aplicação voltou a responder.
---	---

Aprendizado: uma instância pode estar running no console e ainda assim não servir tráfego. O estado da instância e o estado do serviço são camadas independentes. Em produção, o systemd deve ser configurado para iniciar o serviço automaticamente — `sudo systemctl enable httpd` — evitando que uma reinicialização de instância derrube o serviço silenciosamente.

6.3 Segurança por design: Instance Profile vs. credenciais estáticas

Ao associar o perfil IAM Bastion-Role à instância, ela recebeu permissão para chamar APIs da AWS sem nenhuma chave de acesso configurada manualmente. As credenciais são temporárias, rotacionadas automaticamente pelo serviço de metadados da instância (IMDS) e nunca ficam gravadas em disco.

Por que isso importa: Credenciais estáticas configuradas via `aws configure` ficam gravadas em `~/.aws/credentials`. Se a instância for comprometida, o atacante herda essas credenciais indefinidamente. Com Instance Profile, o tempo de vida da credencial é de horas — não de anos.

6.4 Parâmetros dinâmicos como padrão de automação

Recuperar AMI ID, Subnet ID e Security Group ID via CLI em tempo de execução — em vez de fixá-los no script — foi uma das decisões mais relevantes do projeto. Scripts com valores hardcoded falham silenciosamente quando migrados entre contas, regiões ou ambientes. A recuperação dinâmica transforma um script de laboratório em algo que poderia entrar em um pipeline de CI/CD sem modificação.